Building a RAG System for Employee Cybersecurity

IT7075 Applied AI for Cybersecurity | Mini Project Report

1. How the system works, end to end

My project uses retrieval-augmented generation (RAG) to answer employee cybersecurity questions using a small collection of documents I created. Instead of relying on general information alone, the program first searches my documents and then returns text supported by a source file. The implementation is a simple, offline, extractive RAG prototype: it selects a sentence from retrieved material rather than using a separate large language model to compose a new answer.

Chunking the documents

I first load five Markdown files from rag_project/my_kb, excluding SOURCES.md, and verify that the evidence phrases for the answerable questions appear in the documents. I split each document into pieces of up to 600 characters with 100 characters of overlap. This produced 28 chunks. Overlap helps preserve information that might otherwise be split between two pieces. If chunks are too large, retrieval may bring back a lot of unrelated text; if they are too small, important context or even a sentence can be broken apart. My fixed character boundaries did sometimes cut sentences short, which later affected answer quality.

Embeddings and the vector database

An embedding is a list of numbers representing the meaning of a piece of text. Texts with similar meanings can have nearby representations even if they do not share the exact same words. I use the offline all-MiniLM-L6-v2 model to embed the chunks and user questions. ChromaDB stores each chunk with its embedding, an ID, and source-file metadata. For a question, it returns matching chunk text, the source filename, and a distance score. Lower raw distance generally means a closer vector match in this setup.

The three RAG stages

Retrieve: The question is embedded and ChromaDB searches for relevant chunks. I later improved retrieval by searching up to ten chunks and reranking them using keyword overlap. Augment: The selected chunk and its source are used as the evidence for the answer. Generate: My function chooses a sentence from the highest-ranked chunk, attaches its source, or refuses when the raw distance is above a cutoff. This last stage is extraction rather than full language-model generation.

Notebook locations: The original course exercises are labeled TODO 1-5. My own project starts after “Part 6: from here on, the code is yours.” The later cells load the files and gold set; define make_chunks; build my_collection; define retrieve_documents and rag_answer; and produce the experiment table, chart, and answer review.

Knowledge Base and Test Questions

2. What I built

I chose cybersecurity best practices for employees because it is practical and specific enough to test. My five original, AI-assisted educational documents cover password security, phishing awareness, multifactor authentication, safe browsing, and cybersecurity incident reporting. I recorded their origin in SOURCES.md. These are instructional examples, not verified company policies or real incident records.

The eight-question gold set

No.	Question	In KB?
1	Why should employees avoid reusing work passwords?	Yes
2	What should employees do if they receive an unexpected financial request?	Yes
3	What is an MFA fatigue attack?	Yes
4	Does HTTPS guarantee that a website is legitimate?	Yes
5	What information should employees include in a cybersecurity incident report?	Yes
6	What should employees do after receiving an unexpected MFA prompt?	Yes
7	What is the capital of France?	No
8	What exact dollar amount did the company lose in a ransomware attack in 2025?	No

Why include questions the documents cannot answer?

The France question is clearly outside the employee cybersecurity topic. The ransomware-loss question is closer to cybersecurity, but asks for an exact company-specific number that my documents never provide. I included both to see whether the system would acknowledge missing evidence rather than invent an answer. The second is especially useful because a system could sound plausible while making up a dollar amount.

How I checked the test data

The goldset.json file contains six answerable questions with short evidence phrases and two unanswerable questions with empty evidence lists. My notebook checks that the evidence phrases occur in the combined knowledge-base text. That check confirms the information exists somewhere in the documents; it does not guarantee retrieval will select the correct chunk or that the final sentence will be complete.

Experiments and Results

3. Comparing retrieval distance cutoffs

I varied one setting, the refusal cutoff, while keeping the documents, embeddings, and keyword-reranked retrieval procedure the same. The cutoff decides whether the system answers using its highest-ranked chunk or refuses. I measured answer/refusal decisions for all eight questions. These are not independent factual-accuracy scores.

Cutoff	Answered / 6	Correct refusals / 2	Wrong refusals	Wrong answers	Correct decisions / 8
0.75	2	2	4	0	4
0.85	4	2	2	0	6
1.0	5	2	1	0	7
1.2	5	2	1	0	7
1.35	6	2	0	0	8
1.5	6	1	0	1	7

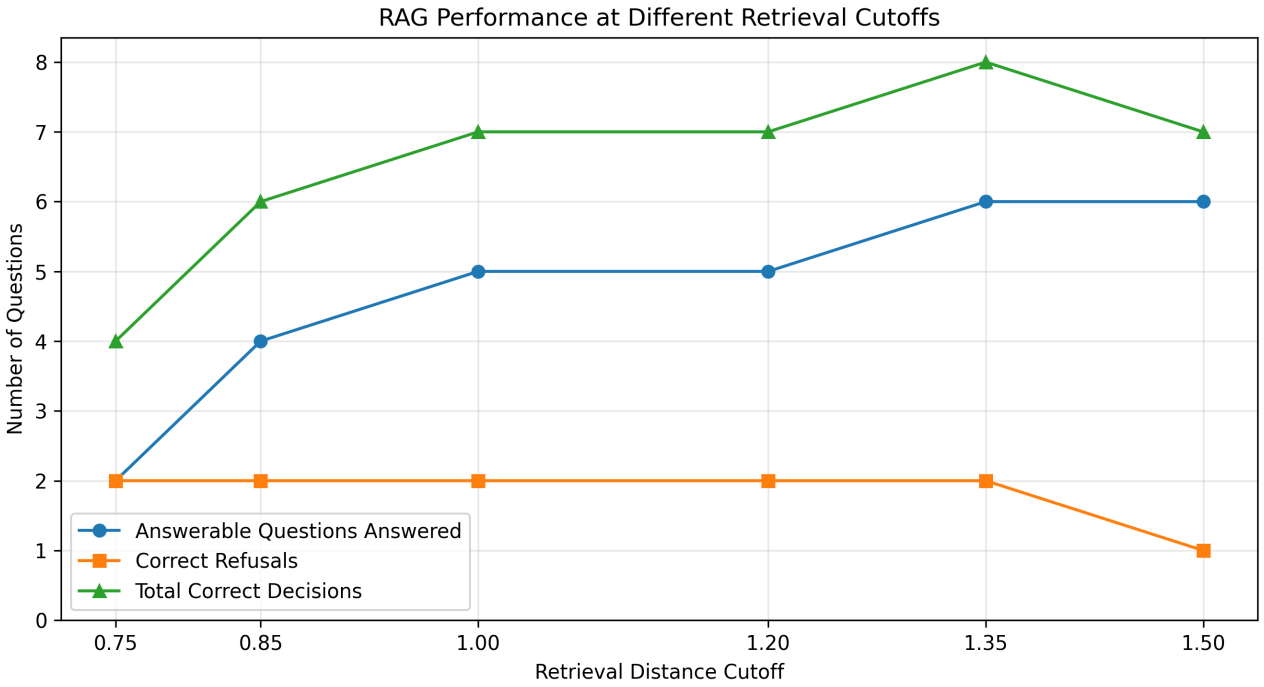


Figure 1. Cutoff experiment chart saved by the notebook as cutoff_experiment.png.

Interpreting the Experiment

Which setting mattered most?

The distance cutoff made the largest measured difference in this experiment. At 0.75, the system answered only two of the six answerable questions; four were incorrectly refused. At 1.35, it answered all six and refused both unsupported questions, giving 8/8 correct answer/refusal decisions. Raising the cutoff to 1.50 caused the ransomware-dollar question to receive an answer when it should have been refused. This demonstrates a tradeoff: a strict cutoff can reject useful evidence, while a loose cutoff can allow unsupported answers.

What settings I would use

For this particular knowledge base and eight-question test set, I would start with 600-character chunks, 100-character overlap, the all-MiniLM-L6-v2 embedding model, ChromaDB, keyword reranking, and a 1.35 refusal cutoff. The reranker was also important: it corrected an early retrieval failure. However, 1.35 was selected after inspecting these same questions, so it should not be treated as a universally safe threshold. I would retest it on new questions before using it in a real workplace.

4. One question the system initially got wrong

My clearest mistake was: “What should employees do if they receive an unexpected financial request?” The first retrieval method returned incident_reporting.md with a raw distance of 1.166. With the original 0.85 cutoff, the system refused even though the answer was present in phishing_awareness.md. This was an incorrect refusal caused by retrieval not finding the right evidence within the allowed distance.

I changed retrieval to consider up to ten vector-search results and rerank them based on shared question keywords. The top result then came from phishing_awareness.md, with raw distance 1.313. The larger distance is not a sign of improved vector similarity: it reflects that keyword reranking promoted a more relevant chunk even though its original vector distance was higher. After testing all eight questions, a 1.35 cutoff allowed this answer while still refusing the two unsupported questions.

An answer-quality mistake that remained

Even when the system made the correct answer/refusal decision, it did not always produce a complete answer. For the incident-reporting question, the saved output ends mid-word: “the affected device or account, a descr”. That happened because character-based chunking cut the source sentence, and the extractive answer function copied the truncated text. A better version would split on sentence or paragraph boundaries and verify that a selected answer is complete.

Limits, Lessons, and Next Steps

5. What my evidence does not tell me

The evaluation set is very small. There are only six answerable questions, so one additional wrong decision changes the answerable-question rate by about 16.7 percentage points. The two unanswerable questions are also too few to establish that the system reliably refuses unsupported requests. My best 8/8 result describes these eight questions, not how the system would perform across unfamiliar wording, other security topics, or actual workplace situations.

I also used the same eight questions to choose the cutoff and then report its performance. This makes the result optimistic because the configuration was tuned to the test set. A stronger evaluation would reserve a separate group of unseen questions, include paraphrases and misleading near-matches, and check whether the selected chunks actually contain the required evidence.

What the score measures - and what it misses

My experiment counts a question as successful if the system answered an answerable question or refused an unanswerable one. It does not automatically judge whether an answer is factually correct, complete, or clear. The incident-reporting answer demonstrates that gap. I manually inspected the saved answer review and found relevant sources for the visible answerable cases, but a full evaluation would score answer correctness and evidence support separately.

Additional limitations

The knowledge base contains five short, AI-assisted documents on a narrow topic. They are not authoritative organizational policies and may omit important exceptions. The program does not use a full generative language model in its custom answer stage; it selects a sentence, which can retain Markdown headings, line breaks, and clipped fragments. Its keyword reranking is simple, and the cutoff is applied to the raw vector distance after reranking rather than to the combined ranking score. This can make a keyword-promoted chunk pass or fail differently than its ranking position might suggest.

What I would improve next

I would use sentence-aware chunking, preserve full paragraphs around evidence, and add a check for truncated responses. I would also test more diverse questions, including questions that look related but are not answered in the documents. Finally, I would compare the extractive approach with a small local language model instructed to answer only from cited passages and refuse when the evidence is missing.

Conclusion

This project helped me understand that RAG is more than retrieving text. The quality of the chunks, the embedding search, the reranking method, and the refusal threshold all affect the final answer. My prototype handled the eight answer/refusal decisions correctly at the selected setting, but the truncated incident-reporting response showed why evaluation must examine answer quality as well as a numerical score.